Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
High Performance Computing (CSDS125)
Lab Assignment: Parallel Linear Regression using CUDA and OpenMP

Objective

To implement linear regression with two input features using gradient descent, and compare the execution time of sequential CPU, OpenMP CPU, and CUDA GPU implementations.

Method

The model predicts the output using

$$\hat{y} = w_1 x_1 + w_2 x_2 + b.$$

For each data point, the error is $e_i = \hat{y}_i - y_i$. The gradients are

$$\frac{\partial J}{\partial w_1} = \frac{2}{N}\sum_{i=1}^{N} e_i x_{1i}, \quad \frac{\partial J}{\partial w_2} = \frac{2}{N}\sum_{i=1}^{N} e_i x_{2i},$$

$$\frac{\partial J}{\partial b} = \frac{2}{N}\sum_{i=1}^{N} e_i.$$

Each parameter is updated using $w \leftarrow w - \alpha \nabla J$, where $\alpha = 0.1$. The experiment uses $N = 1{,}000{,}000$ data points and 300 iterations. The target values are generated as $y = 3x_1 + 2x_2 + 1$.

1. Sequential and OpenMP Implementation

The same training function is used for both CPU versions. OpenMP parallelizes the data-point loop and combines gradient contributions with a reduction.

```
#include <iostream>
#include <chrono>
#include <omp.h>
using namespace std;
using namespace chrono;

const int N=1000000, ITER=300;
const double LR=0.1;

double x1(int i){ return (i%1000)/1000.0; }
double x2(int i){ return ((i*7)%1000)/1000.0; }
double y(int i){ return 3*x1(i)+2*x2(i)+1; }

void train(bool parallel){
    double w1=0, w2=0, b=0;
    auto start=high_resolution_clock::now();

    for(int it=0; it<ITER; it++){
        double dw1=0, dw2=0, db=0;

```

```
        if(parallel){
            #pragma omp parallel for reduction(+:dw1,dw2,db)
            for(int i=0; i<N; i++){
                double e=w1*x1(i)+w2*x2(i)+b-y(i);
                dw1+=e*x1(i);
                dw2+=e*x2(i);
                db+=e;
            }
        } else {
            for(int i=0; i<N; i++){
                double e=w1*x1(i)+w2*x2(i)+b-y(i);
                dw1+=e*x1(i);
                dw2+=e*x2(i);
                db+=e;
            }
        }

        w1-=LR*2*dw1/N;
        w2-=LR*2*dw2/N;
        b -=LR*2*db/N;
    }

    auto end=high_resolution_clock::now();
    cout << (parallel ? "OpenMP" : "Sequential")
         << " time: "
         << duration<double,milli>(end-start).count() << " ms"
         << " | w1=" << w1 << " w2=" << w2
         << " b=" << b << endl;
}

int main(){
    cout << "Data points: " << N
         << ", Iterations: " << ITER << "\\n";
    train(false);
    train(true);
}
```

2. CUDA Implementation

CUDA assigns data points to GPU threads. Each block reduces its local gradient values, and a second kernel sums the block-level partial gradients and updates the model parameters. Both kernels are launched in the same stream, so each update completes before the next iteration's gradient kernel executes.

```
#include <iostream>
#include <chrono>
#include <cuda_runtime.h>
using namespace std;
using namespace chrono;

const int N=1000000, ITER=300, BLOCK=256;
const double LR=0.1;

__device__ double fx1(int i){ return (i%1000)/1000.0; }
__device__ double fx2(int i){ return ((i*7)%1000)/1000.0; }
__device__ double fy(int i){
    return 3*fx1(i)+2*fx2(i)+1;
}

__global__ void gradients(double *w1,double *w2,double *b,
                          double *p1,double *p2,double *pb){
    __shared__ double s1[BLOCK],s2[BLOCK],sb[BLOCK];
    int i=blockIdx.x*blockDim.x+threadIdx.x;
    int t=threadIdx.x;
    double a=0,c=0,d=0;

    if(i<N){
```

```
        double e=(*w1)*fx1(i)+(*w2)*fx2(i)+(*b)-fy(i);
        a=e*fx1(i); c=e*fx2(i); d=e;
    }
    s1[t]=a; s2[t]=c; sb[t]=d;
    __syncthreads();

    for(int step=BLOCK/2;step>0;step/=2){
        if(t<step){
            s1[t]+=s1[t+step];
            s2[t]+=s2[t+step];
            sb[t]+=sb[t+step];
        }
        __syncthreads();
    }
    if(t==0){
        p1[blockIdx.x]=s1[0];
        p2[blockIdx.x]=s2[0];
        pb[blockIdx.x]=sb[0];
    }
}

__global__ void update(double *p1,double *p2,double *pb,
                       int blocks,double *w1,double *w2,double *b){
    if(blockIdx.x==0 && threadIdx.x==0){
        double a=0,c=0,d=0;
        for(int i=0;i<blocks;i++){
            a+=p1[i]; c+=p2[i]; d+=pb[i];
        }
        *w1-=LR*2*a/N;
        *w2-=LR*2*c/N;
        *b -=LR*2*d/N;
    }
}

int main(){
    int blocks=(N+BLOCK-1)/BLOCK;
    double *p1,*p2,*pb,*w1,*w2,*b;
    cudaMallocManaged(&p1,blocks*sizeof(double));
    cudaMallocManaged(&p2,blocks*sizeof(double));
    cudaMallocManaged(&pb,blocks*sizeof(double));
    cudaMallocManaged(&w1,sizeof(double));
    cudaMallocManaged(&w2,sizeof(double));
    cudaMallocManaged(&b,sizeof(double));

    *w1=0; *w2=0; *b=0;
    cudaDeviceSynchronize();
    auto start=high_resolution_clock::now();

    for(int it=0;it<ITER;it++){
        gradients<<<blocks,BLOCK>>>(w1,w2,b,p1,p2,pb);
        update<<<1,1>>>(p1,p2,pb,blocks,w1,w2,b);
    }
    cudaDeviceSynchronize();
    auto end=high_resolution_clock::now();

    cout << "CUDA time: "
         << duration<double,milli>(end-start).count() << " ms\\n";
    cout << "w1 = " << *w1 << ", w2 = " << *w2
         << ", b = " << *b << endl;

    cudaFree(p1); cudaFree(p2); cudaFree(pb);
    cudaFree(w1); cudaFree(w2); cudaFree(b);
}
```

3. Experimental Output

The following values were observed in Google Colab. Execution times can vary with the available CPU/GPU and system load.

CPU output

```
Data points: 1000000, Iterations: 300
Sequential time: 1784.74 ms | w1=2.96416 w2=1.9778 b=1.03092
OpenMP time: 1451.68 ms | w1=2.96416 w2=1.9778 b=1.03092
```

CUDA output

```
CUDA time: 219.711 ms
w1 = 2.96416, w2 = 1.9778, b = 1.03092
```

4. Performance Comparison

Implementation	Time (ms)	Speedup vs. sequential
Sequential CPU	1784.74	1.00×
OpenMP CPU (4 threads)	1451.68	1.23×
CUDA GPU	219.711	8.12×

OpenMP speedup is $1784.74/1451.68 \approx 1.23$. CUDA speedup is $1784.74/219.711 \approx 8.12$. Based on these observed timings, CUDA took about 87.7% less time than the sequential version. The learned parameters are approximately $w_1 = 2.964$, $w_2 = 1.978$, and $b = 1.031$, which are close to the generating values 3, 2, and 1.

5. Short Analysis and Conclusion

The experiment shows that parallel gradient computation reduces training time for one million data points over 300 iterations. OpenMP provides a modest improvement on the CPU, while CUDA achieves the best measured runtime because many GPU threads process data points concurrently. All implementations converge to approximately the same parameters, providing a consistency check. The timings are specific to this Colab session and include CUDA kernel-launch overhead; repeated runs and GPU warm-up can improve the reliability of the comparison.